

PROJEKT DATA SCIENCE

Umsatzprognose mit KNIME

Analyse, Bereinigung und Modellierung von Filialdaten

Technische Hochschule Mittelhessen | SoSe 2026

Gruppe: Kana Tezo, Moumani Tchokote, Pokem Tezo, Mamo Tiotsap

Dozent: Frank Kammer



Überblick der Präsentation

Vom Rohdatensatz bis zur Interpretation des Prognosemodells

01

Datengrundlage

02

Datenbereinigung

03

DQ-Kennzahlen

04

XGBoost

05

Modellierung

06

Fehleranalyse

07

Clustering & Fazit

01

Zwei Dateien bilden die Grundlage

Am Anfang standen zwei Tabellen mit unterschiedlichen Aufgaben:

filialumsatz.csv

enthält die Tageswerte der Filialen, und

filialen.csv

enthält die festen Eigenschaften der Filialen.



filialumsatz.csv

336 084

Zeilen

9

Spalten

Tageswerte



filialen.csv

406

Zeilen

10

Spalten

Stammdaten

filialen.csv beschreibt die Filialen selbst

Hier stehen die Eigenschaften, die nicht jeden Tag wechseln.



406

Filialen im Stammdatensatz

10 Spalten

Typ, Sortiment, Wettbewerb und
Langzeitpromo



Store

Schlüssel zur Umsatzdatei



StoreType

Typ der Filiale, z. B. a, b, c, d



Assortment

Art des Sortiments



CompetitionDistance

Distanz zur Konkurrenz



CompetitionOpenSince

Monat und Jahr der Konkurrenzöffnung



Promo2

Teilnahme an Langzeitpromotion



Promo2SinceWeek

Startwoche der Langzeitpromo



Promo2SinceYear

Startjahr der Langzeitpromo



PromoInterval

Monate, in denen Promo2 läuft

filialen.csv im ersten Blick

Schon wenige Zeilen zeigen, warum die Stammdaten zuerst bereinigt werden mussten.

Store	StoreType	Assortment	Comp.Distance	Comp.Month	Comp.Year	Promo2	PromoInterval
104	Type_A	basic	450	03	2012	1	January, Apr, JUL, October
104	a	Basic assortment	450	3	2012	1	JAN, apr, 07, oct
494	store type b	assortment_a	-	-	-	0	kein PromoInterval
554	typ c	EXTENDED	-1	13	2011	1	Feb, 05, AUG, nov
754	Typ A	Basic	999999	-	-	1	-
1029	StoreType_A	A	1200	Mai	2014	-	no interval
138	-	Extra	300	02	2010	-	kein PromoInterval

Typische Probleme: doppelte Store-IDs, uneinheitliche Schreibweisen sowie fehlende oder unplausible Angaben in Konkurrenz- und Promo-Spalten.

Store

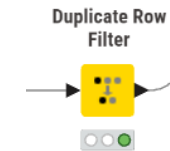
Store ist der Schlüssel zwischen den Filialdaten und den Tagesumsätzen.

Tabellenbeispiel

Store	StoreType	Assortment	PromotInterval
104	Type_A	basic	none
104	a	basic	no interval
494	store type b	assortment_a	-
494	Typ B	a	kein PromotInterval
554	typ c	c	-
554	store type c	EXTENDED	none
1029	Typ A	A	-
1029	Typ A	Basic	none

0 fehlende Store-Werte, aber 4 doppelte Store-IDs.

KNIME-Node



Idee

Zuerst wurden doppelte Store-IDs erkannt. Danach wurden diese Fälle mit einem Row Filter aus dem finalen Stammdatensatz entfernt.

Ergebnis

Nach der Bereinigung bleibt pro Store nur ein gültiger Stammdatensatz.

StoreType

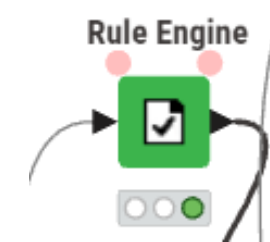
StoreType beschreibt den Typ der Filiale.

Tabellenbeispiel

Store	StoreType roh	StoreType_clean
2	StoreType_A	a
12	A	a
17	typ a	a
36	Typ A	a
46	Typ C	c
104	Type_A	a
494	store type b	b
554	store type c	c
1029	Typ A	a

StoreType hatte viele Schreibweisen, obwohl fachlich nur die Typen a, b, c und d gebraucht wurden.

KNIME-Node



Idee

Wir haben gesehen, dass derselbe Filialtyp unterschiedlich geschrieben wurde. Deshalb wurden die Texte vereinheitlicht und danach klar codiert.

Ergebnis

Alle Varianten wurden auf a, b, c und d reduziert.

Assortment

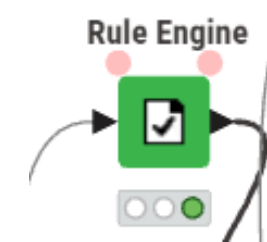
Assortment beschreibt die Sortimentsart einer Filiale.

Tabellenbeispiel

Store	Assortment roh	Assortment_clean
1	Type_A	a
2	Typ A	a
7	EXTENDED	c
15	Extended assortment	c
36	assortment_c	c
46	type_a	a
104	basic	a
494	assortment_a	a
1029	Basic	a

Assortment hatte viele Schreibweisen, obwohl fachlich nur die Kategorien a, b und c gebraucht wurden.

KNIME-Node



Idee

Auch beim Sortiment gab es mehrere Schreibweisen für dieselbe Bedeutung. Die Werte wurden zuerst bereinigt und dann auf die gültigen Kategorien abgebildet.

Ergebnis

Die Sortimentswerte wurden auf a, b und c vereinheitlicht.

CompetitionDistance

CompetitionDistance beschreibt die Entfernung zur nächsten Konkurrenzfiliale.

Tabellenbeispiel

Store	roh	final
18	-	5.540
23	-	3.370
333	1.991.100	2.840
727	773.807	1.420
1026	206.671	1.190
1040	-	1.420
7	24.000	24.000
49	18.010	18.010
17	50	50

21 fehlende Distanzen und 3 extrem große Ausreißer bis 1.991.100.

KNIME-Node



Idee

Bei der Distanz gab es leere und unrealistisch große Werte. Wir haben diese Fälle markiert und mit passenden Medianwerten ersetzt.

Ergebnis

Fehlende und extreme Werte wurden über Filter, GroupBy und Joiner plausibel ersetzt.

CompetitionOpenSinceMonth / Year

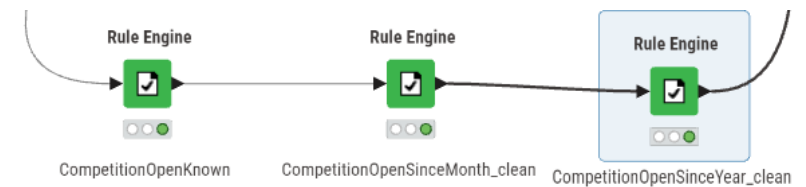
Monat und Jahr beschreiben gemeinsam, seit wann eine Konkurrenzfiliale existiert.

Tabellenbeispiel

Store	Month roh	Year roh	Known	clean
12	-	-	0	0 / 0
26	-	-	0	0 / 0
62	-	-	0	0 / 0
79	-	-	0	0 / 0
18	6	2010	1	6 / 2010
23	8	2005	1	8 / 2005
47	4	2013	1	4 / 2013
103	5	2015	1	5 / 2015

132 Filialen hatten weder CompetitionOpenSinceMonth noch CompetitionOpenSinceYear.

KNIME-Node



Idee

Monat und Jahr ergeben nur zusammen eine sinnvolle Information. Wenn beide fehlten, wurde kein künstliches Datum erfunden, sondern der Status markiert.

Ergebnis

Monat und Jahr wurden gemeinsam geprüft und bereinigt.

Promo2-Informationen

Diese Spalten beschreiben, ob und seit wann eine Langzeitpromotion aktiv ist.

Tabellenbeispiel

Store	Promo2	Week	Year	PromoInterval roh
1	0	-	-	-
2	1	13	2010	January, Apr, JUL, October
29	0	-	-	kein PromoInterval
46	1	14	2011	JAN, apr, 07, oct
78	1	5	2013	Feb, 05, AUG, nov
102	1	10	2014	März, Jun., Sept, Dezember
128	1	1	2013	Januar, APR, Jul, 10
138	-	-	-	kein PromoInterval
754	1	10	2014	-

KNIME-Node



Idee

Promo2, Startwoche, Startjahr und Intervall hängen fachlich zusammen. Deshalb wurden sie nicht einzeln, sondern im Zusammenhang geprüft.

Ergebnis

Die vier Promo-Spalten wurden gemeinsam geprüft.

Promo2-Informationen waren über vier Spalten verteilt und teilweise widersprüchlich oder uneinheitlich geschrieben.

PromoInterval

PromoInterval beschreibt die Monate, in denen die Langzeitpromotion läuft.

Tabellenbeispiel

Store	PromoInterval roh	PromoInterval_clean
2	January, Apr, JUL, October	Jan, Apr, Jul, Oct
46	JAN, apr, 07, oct	Jan, Apr, Jul, Oct
128	Januar, APR, Jul, 10	Jan, Apr, Jul, Oct
78	Feb, 05, AUG, nov	Feb, May, Aug, Nov
145	February, MAY, 08, NOV	Feb, May, Aug, Nov
102	März, Jun., Sept, Dezember	Mar, Jun, Sept, Dec
29	kein PromoInterval	no_interval
23	no interval	no_interval
754	-	Jan, Apr, Jul, Oct

KNIME-Node



Idee

Bei PromoInterval waren Sprache, Großschreibung und Monatsformate gemischt. Wir haben die Monate normalisiert und auf feste Intervallmuster gebracht.

Ergebnis

Die Intervalle wurden auf vier gültige Formen reduziert.

PromoInterval hatte viele Schreibweisen und musste aus dem Kontext von Promo2 interpretiert werden.

Promo2SinceWeek / Promo2SinceYear

Woche und Jahr beschreiben den Start der Langzeitpromotion.

Tabellenbeispiel

Store	P2 roh	W roh	Y roh	P2 clean	W/Y clean
1	0	-	-	0	0 / 0
2	1	13	2010	1	13 / 2010
23	0	-	-	0	0 / 0
46	1	14	2011	1	14 / 2011
78	1	5	2013	1	5 / 2013
102	1	10	2014	1	10 / 2014
138	-	-	-	0	0 / 0
754	1	10	2014	1	10 / 2014

KNIME-Node



Idee

Wenn Promo2 nicht aktiv war, durften Week und Year nicht wie echte Startdaten wirken. Deshalb wurden diese Fälle auf 0 gesetzt.

Ergebnis

Startwoche und Startjahr passen jetzt zu Promo2_clean.

Promo2 hatte 4 fehlende Werte; Week und Year fehlten in 223 Zeilen und mussten abhängig von Promo2 behandelt werden.

filialen.csv bereinigt

Nach der Bereinigung sind die Stammdaten einheitlich und für den Join nutzbar.

Store	StoreType_clean	Assortment_clean	Comp.Distance_final	Comp.Known	Promo2_clean	PromoInterval_clean
23	d	a	3370	1	0	no_interval
26	d	a	2300	0	0	no_interval
58	a	c	7340	1	1	Jan,Apr,Jul,Oct
62	a	a	2050	0	0	no_interval
73	a	a	1650	1	1	Jan,Apr,Jul,Oct
90	d	a	3300	0	0	no_interval
102	a	a	150	1	1	Mar,Jun,Sept,Dec

Eindeutig

Jede Store-ID bleibt nur einmal im Stammdatensatz.

Standardisiert

StoreType, Assortment und PromoInterval haben feste Kategorien.

Modellbereit

Die Spalten können später mit den Tagesumsätzen verbunden werden.

filialumsatz.csv zeigt jeden Verkaufstag

Diese Datei beschreibt die täglichen Umsätze und direkten Einflussgrößen.



336 084

Tagesbeobachtungen

9 Spalten

eine Zeile pro Filiale und Tag



Store

eindeutige Nummer der Filiale



Date

Datum der Beobachtung



DayOfWeek

Wochentag als Zahl



Sales

Tagesumsatz und spätere Zielvariable



Customers

Anzahl der Kunden am Tag



Open

Filiale offen oder geschlossen



Promo

aktive Tagespromotion



StateHoliday

staatlicher Feiertag



SchoolHoliday

Schulferien oder nicht

filialumsatz.csv im ersten Blick

Einige Zeilen zeigen direkt, warum die Tagesdaten vor der Modellierung bereinigt werden mussten.

Store	Date	DayOfWeek	Sales	Customers	Open	Promo	StateHoliday	SchoolHoliday
132	2015-07-31	5	10822	774	1.0	-	0	0.0
166	2015-07-31	5	-	961	1.0	1.0	No Holiday	1.0
347	2015-07-31	5	7820	525	1.0	-	None	1.0
505	2015-07-31	5	6998	725	-	1.0	no holiday	1.0
647	2015-07-31	5	523827	46674	1.0	1.0	no holiday	1.0
1025	2015-07-31	5	9964	-	1.0	1.0	None	1.0
93	2015-07-30	4	-882939	598	1.0	1.0	no holiday	1.0
150	2015-07-30	4	12007	1070	0.0	1.0	-	1.0
4	2015-07-31	5	14215	1522	1.0	1.0	None	1.0

Fehlende Werte

Leere Felder kamen z. B. bei Sales, Customers, Open, Promo und StateHoliday vor.

Ausreißer

Einige Sales- und Customers-Werte waren für normale Verkaufstage extrem hoch.

Schreibweisen

StateHoliday erschien in mehreren Varianten und musste vereinheitlicht werden.

Sales / Customers / Open

Diese Spalten beschreiben Umsatz, Kundenanzahl und Öffnungsstatus pro Tag.

Tabellenbeispiel

Store	Date	Sales	Cust.	Open	Auffälligkeit
102	2015-07-31	15409	1494	0	Umsatz trotz geschlossen
166	2015-07-31	-	961	1	Sales fehlt
1025	2015-07-31	9964	-	1	Customers fehlt
281	2015-07-30	-6669	520	1	negativer Umsatz
301	2015-07-28	999999	99999	1	Dummy-Werte
388	2015-07-29	4164	941	0	Open-Widerspruch
451	2015-07-26	0	0	-	Open fehlt
578	2015-07-26	7090	-	1	Customers fehlt

Idee

Sales, Customers und Open wurden zusammen betrachtet, weil Fehler oft erst durch die Kombination der drei Spalten sichtbar wurden.

Ergebnis

Die finalen Spalten sind für Analyse und Modellierung nutzbar.

Sales, Customers und Open enthielten fehlende Werte, Ausreißer und logische Widersprüche zwischen den Spalten.

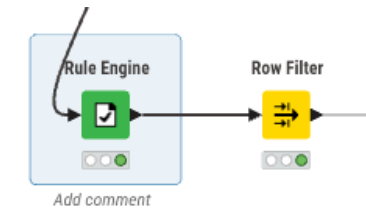
Sales

Sales ist die wichtigste Zielvariable für das spätere Modell.

Tabellenbeispiel

Store	Date	Sales	Open	Customers
166	2015-07-31	-	1	961
281	2015-07-30	-6669	1	520
301	2015-07-28	999999	1	99999
102	2015-07-31	15409	0	1494
451	2015-07-26	0	-	0
578	2015-07-26	7090	1	-
64	2015-07-25	6394	1	682

KNIME-Workflow



Idee

Sales ist die Zielvariable. Deshalb wollten wir hier keine künstlichen Werte erzeugen, sondern nur plausible echte Umsätze behalten.

Ergebnis

Da Sales die Zielvariable ist, wurden keine künstlichen Werte erzeugt. Fehlende, negative und unrealistisch hohe Sales-Zeilen machten nur einen kleinen Teil der Daten aus und wurden aus dem finalen Datensatz entfernt.

Entscheidung: Sales nicht ersetzen, sondern problematische Zeilen entfernen.

Sales / Customers / Open

Die drei Spalten wurden gemeinsam betrachtet.

Tabellenbeispiel

Store	Date	Sales_clean	Customers	Open	final
578	2015-07-26	7090	-	1	Customers ergänzen
1025	2015-07-31	9964	-	1	Customers ergänzen
388	2015-07-29	4164	941	0	Open = 1
102	2015-07-31	15409	1494	0	Open = 1
451	2015-07-26	0	0	-	Open = 0
64	2015-07-25	6394	682	1	bleibt

Prinzip: Zwei passende Werte bestimmen den dritten.

Idee

Die drei Spalten wurden zusammen geprüft, weil eine fehlende oder falsche Angabe oft durch die beiden anderen erklärbar war.

Ergebnis

Wenn zwei Informationen logisch zusammenpassten, wurde daraus die dritte Information abgeleitet.

Beispiel: Wenn Sales_clean vorhanden ist und Open = 1 gilt, konnte eine fehlende Kundenanzahl über einen Median ergänzt werden.

Wenn Sales_clean und Customers vorhanden waren, aber Open = 0 stand, wurde Open auf 1 korrigiert.

Promo, StateHoliday und SchoolHoliday

Die drei Spalten wurden einheitlich normalisiert und fehlende Werte wurden auf 0 gesetzt.

Promo

Store	Date	Original	Clean
12	2015-07-31	1.0	1
19	2015-07-31	0.0	0
45	2015-07-30	-	0
103	2015-07-29	1	1
211	2015-07-28	0	0

StateHoliday

Store	Date	Original	Clean
7	2015-04-03	a	a
18	2015-04-06	Typ A	a
26	2014-12-25	B	b
41	2015-07-31	No Holiday	0
58	2015-07-30	kein Feiertag	0

SchoolHoliday

Store	Date	Original	Clean
12	2015-07-31	1.0	1
19	2015-07-31	0.0	0
45	2015-07-30	-	0
103	2015-07-29	1	1
211	2015-07-28	0	0

filialumsatz.csv bereinigt

Nach der Bereinigung bleiben nur plausible Tageswerte für Analyse und Modellierung.

Store	Date	Sales	Customers_final	Open_clean	Promo_clean	StateHoliday_clean	SchoolHoliday_clean
23	2015-07-31	7238	557	1	1	0	1
26	2015-07-31	7406	579	1	1	0	1
58	2015-07-31	7855	552	1	1	0	0
62	2015-07-31	7129	734	1	1	0	1
73	2015-07-31	6026	569	1	1	0	1
90	2015-07-31	8074	633	1	1	0	1
102	2015-07-31	15409	1494	1	1	0	1

Keine Problemcodes

Fehlende Werte und extreme Ausreißer wurden behandelt.

Klare Variablen

Open, Promo und Feiertage liegen als saubere Modellspalten vor.

Weiterverwendung

Diese Tabelle wurde für DQ-Kennzahlen, XGBoost und Clustering genutzt.

Join der beiden Tabellen

Nach der Bereinigung wurden Filialstammdaten und Tagesumsätze zusammengeführt.



Damit wurden nicht alle Stores aus der Umsatzdatei übernommen, sondern nur die Filialen, zu denen auch Stammdaten vorhanden waren.

03

DQ-Kennzahlen für die Filialdaten

Datenqualität wurde messbar gemacht: vor und nach der Bereinigung.

Fünf Kennzahlen

Die Kennzahlen zeigen nicht nur einzelne Fehler, sondern die Qualität des gesamten Datensatzes.

Vollständigkeit

Anteil der vorhandenen (nicht fehlenden) Werte

1 – Fehlend / Gesamt

Eindeutigkeit

Zeilen ohne Duplikate

1 – Duplikate / Zeilen

Konsistenz

Keine logischen Widersprüche
(z.B. kein Umsatz bei geschlossener Filiale)

1 – Widersprüche / Zeilen

Plausibilität

Werte im realistischen Wertebereich
(z.B. keine negativen Umsätze, keine Sales=0 bei Open=1)

Zählung unmöglicher Werte

Standardisierung

Einheitliche Schreibweisen für kategoriale Werte
(z.B. StoreType, Assortment, StateHoliday)

Anzahl Varianten → Sollzahl

Probleme vor und nach der Bereinigung

Die Ergebnisse wurden als Vergleich Rohdaten gegen Clean-Datensatz zusammengefasst.

Kennzahl / Problem	Rohdaten	Clean	Maßnahme
Fehlende Werte filialumsatz.csv	56.459	0	ersetzt / entfernt
Fehlende Werte filialen.csv	841	0	ersetzt / entfernt
Doppelte Store-Date-Einträge	1.125	0	Duplikate gelöscht
Unplausible Sales-Werte	1.218	0	Zeilen entfernt
Konsistenzverletzungen	1.532	0	regelbasiert korrigiert
Uneinheitliche Kategorien	vorhanden	0	standardisiert

Berechnung mit echten Zahlen

Vollständigkeit

filialumsatz.csv

1 - 56.459 / 3.024.756

Vorher

98,13
%

Nachher

100 %

Vollständigkeit

filialen.csv

1 - 841 / 4.060

Vorher

79,29
%

Nachher

100 %

Eindeutigkeit

filialumsatz.csv

1 - 1.125 / 336.084

Vorher

99,67
%

Nachher

100 %

Konsistenz

filialumsatz.csv

1 - 1.532 / 336.084

Vorher

99,54
%

Nachher

100 %

Standardisierung der Kategorien

Viele Schreibweisen wurden auf wenige gültige Codes reduziert.

Spalte	Varianten vorher	Beispiele (Auszug)	Varianten nachher
StoreType	30	"type a", "Typ A", "TYPE_A", "StoreType_a" ...	4 → a, b, c, d
Assortment	23	"Basic", "basic assortment", "BASIC" ...	3 → a, b, c
StateHoliday	10	"Feiertag B", "type_b", "easter holiday" ...	4 → 0, a, b, c

KNIME-Idee: String Manipulation für Schreibweisen und Rule Engine für die finalen Codes.

Detail: 1) String Manipulation → lowerCase() / trim() 2) Rule Engine → LIKE-Operator → einheitliche Codes (a / b / c / d)

04

XGBoost-Verfahren

Was ist XGBoost, wie unterscheidet es sich von Random Forest und warum passt es zu unserem Projekt?

Was ist XGBoost?

eXtreme Gradient Boosting baut viele kleine Entscheidungsbäume nacheinander.

Grundidee

Jeder neue Baum korrigiert die Fehler der vorherigen Bäume.
Die Endvorhersage ist die Summe vieler kleiner Teilvorhersagen.

$$\hat{y} = f_1(x) + f_2(x) + f_3(x) + \dots + f_n(x)$$

	Random Forest	XGBoost
Bäume	parallel	sequenziell
Strategie	Mittelwert	Fehlerkorrektur
Methode	Bagging	Boosting
Genauigkeit	gut	oft besser

Entscheidungsbaum und Boosting

Ein Baum allein ist oft zu ungenau. Boosting reduziert den Fehler Schritt für Schritt.

Beispiel Entscheidungsbaum

Ist die Filiale geöffnet?
Nein -> Sales = 0 EUR
Ja -> Gibt es Promo?
Nein -> ca. 5.000 EUR
Ja -> ca. 8.000 EUR

Problem: Ein einzelner Baum ist zu grob und kann overfitten.

Schritt	Vorhersage	Fehler
Baum 1	6.000 EUR	+2.000
Baum 2	7.800 EUR	+200
Baum 3	7.980 EUR	+20
Baum 1000	8.000 EUR	ca. 0

Wie lernt XGBoost?

Die Zielfunktion verbindet Fehler und Modellkomplexität.

$$\text{obj}(\theta) = L(\theta) + \Omega(\theta)$$

$L(\theta)$ — Training Loss

Misst, wie falsch die Vorhersagen sind.

Formel: $L = \sum (y - \hat{y})^2$

Beispiel: $(y - \hat{y})^2$

Konkretes Beispiel:

Sales real = 8.000 €

Vorhersage = 6.000 €

Fehler² = 4.000.000

→ Je kleiner L, desto besser.

$\Omega(\theta)$ — Regularisierung

Bestraft zu komplexe Bäume.
Verhindert Overfitting:
das Modell lernt nicht auswendig,
sondern generalisiert.

Parameter λ (Lambda)
steuert die Stärke der Strafe.
Standard: $\lambda = 1$

Groß. $\lambda \rightarrow$ einfaches Modell
Klein. $\lambda \rightarrow$ komplexeres Modell

Gradient & Hessian

g_i = Richtung der Korrektur
 $g_i = \hat{y} - y$ (1. Ableitung)

h_i = Größe der Korrektur
 $h_i = 1$ (für MSE)

Optimaler Korrekturwert:
 $w^* = -G / (H + \lambda)$

η (Learning Rate) bremst
den Lernschritt pro Baum.

Warum passt XGBoost zu unserem Projekt?

XGBoost eignet sich wenn ...

- Tabellarische Daten (CSV, Datenbanken)
- Mischung aus numerischen und kategorischen Merkmalen
- Große Datenmengen (> 10.000 Zeilen)
- Nicht-lineare Zusammenhänge zwischen Variablen
- Regressions- oder Klassifikationsaufgaben

Bedingung	Unser Datensatz
Tabellarische Daten	✓ finaler Clean-Datensatz
Numerisch + kategorisch	✓ Sales, Promo, StoreType, Assortment
Große Datenmenge	✓ 119.905 Zeilen
Nicht-linearer Zusammenhang	✓ Umsatz hängt von vielen Faktoren ab
Regression	✓ Zielvariable: Sales

Damit ist XGBoost für die Umsatzprognose gut geeignet.

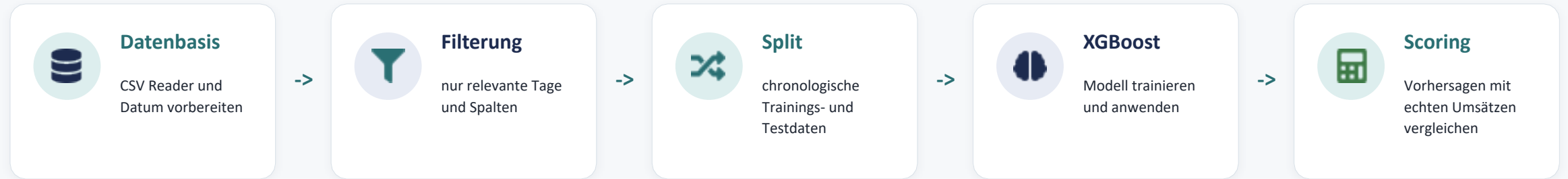
05

Umsatzprognose mit XGBoost

Modellaufbau, Training und Bewertung der Vorhersagen.

KNIME-Workflow: Umsatzprognose

Von der vorbereiteten Tabelle bis zur bewerteten Umsatzprognose.

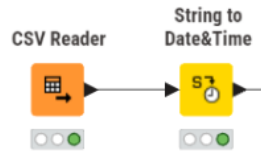


Vorbereitung -> Training -> Prediction -> Bewertung

KNIME-Workflow: Daten einlesen und filtern

Die ersten Knoten bereiten die technische Grundlage für das Modell vor.

CSV Reader + String to Date&Time

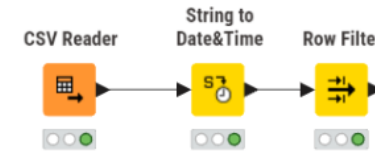


Datensatz einlesen

Zuerst wurde der bereinigte Datensatz eingelesen. Danach wurde Date in ein echtes Datumsformat umgewandelt, damit Zeitinformationen korrekt genutzt werden können.

Daten technisch nutzbar machen

Row Filter



Nur geöffnete Tage

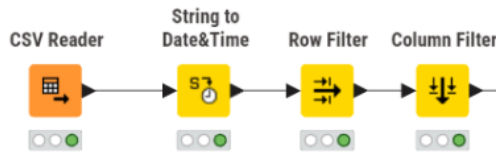
Im Row Filter haben wir nur Tage mit Open_clean = 1 behalten. Geschlossene Tage mit Sales = 0 wurden entfernt, damit das Modell normale Verkaufstage lernt.

119.905 -> 99.483 Zeilen

KNIME-Workflow: Spaltenauswahl und Codierung

Hier wurde entschieden, welche Informationen realistisch für eine Prognose nutzbar sind.

Column Filter

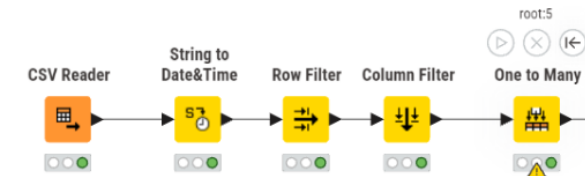


Spalten reduzieren

Store, Open und Customers_final wurden nicht als Eingabevariablen genutzt. Customers kennt man bei einer echten zukünftigen Prognose nicht, und Store sollte nicht nur als ID auswendig gelernt werden.

Store, Open, Customers entfernt

One to Many



Kategorien numerisch machen

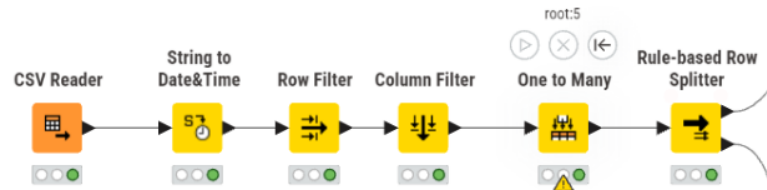
Kategoriale Variablen wie StoreType oder Assortment wurden in numerische Spalten umgewandelt, damit XGBoost sie im Modell verwenden kann.

Text -> Modellspalten

KNIME-Workflow: Split und Modellzweige

Danach wurde der Datensatz für Training und Test strukturiert.

Rule-based Row Splitter



Chronologischer Split

Die Regel war: Date <= 2015-05-31 geht in den Trainingszweig, alle späteren Tage in den Testzweig. So wird die Prognose zeitlich realistischer bewertet.

Train: 92.592 | Test: 6.891

Column Filter nach dem Split



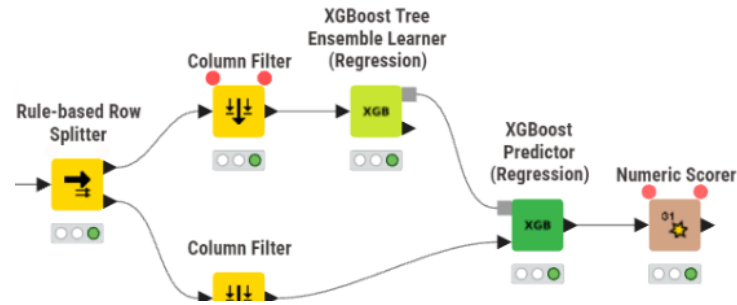
Training und Test vorbereiten

Im Trainingszweig bleibt Sales als Zielvariable vorhanden. Im Testzweig bekommt der Predictor nur die Eingabevariablen und erzeugt daraus die Vorhersage.

zwei Modellzweige

KNIME-Workflow: Training, Prediction und Scoring

Der letzte Teil verbindet Modelltraining, Vorhersage und Bewertung.



XGBoost Learner

Trainiert das Regressionsmodell auf den Trainingsdaten.



XGBoost Predictor

Erzeugt für jede Testzeile eine vorhergesagte Sales-Spalte.



Numeric Scorer

Vergleicht Sales mit Prediction(Sales): R2, MAE, RMSE und MAPE.

Ergebnis: trainiertes XGBoost-Modell + messbare Modellqualität

Modellmetriken: Was sagen die Ergebnisse?

Die Kennzahlen zeigen eine sehr gute Modellgüte, aber auch einzelne größere Fehler.

► 1: Statistics Flow Variables

Rows: 7 | Columns: 1

☐	#	RowID	Prediction (Sales) Number (Float)
☐	1	R^2	0.979
☐	2	mean ε	921.382
☐	3	mean ε	7,595,517.984
☐	4	root mε	2,755.997
☐	5	mean ε	318.572
☐	6	mean ε	0.139
☐	7	adjuste	0.979

0,979

R2

97,9 % erklärte Varianz

921

MAE

durchschnittlicher absoluter Fehler

2.756

RMSE

bestraft große Fehler stärker

13,9 %

MAPE

relativer Fehler im Mittel

0,979

Adjusted R2

auch angepasst sehr hoch



Interpretation

Das Modell ist insgesamt stark. Der höhere RMSE zeigt aber, dass einzelne Extremwerte deutlich größere Fehler erzeugen.

06

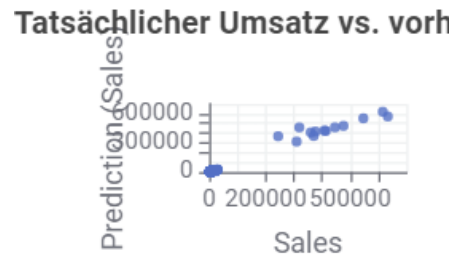
Analyse der Vorhersagefehler

Die Prognosen werden grafisch und nach Gruppen bewertet.

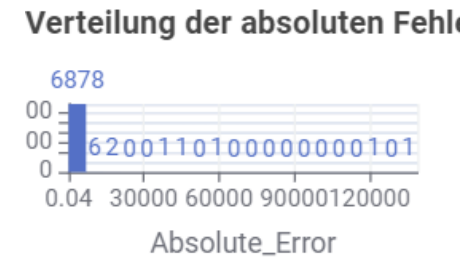
Vorhersagen und Fehler verstehen

Die Grafiken zeigen, wo das Modell stabil ist und wo es schwieriger wird.

Tatsächlicher vs. vorhergesagter Umsatz



Verteilung der absoluten Fehler



Stabil im Normalbereich

Bei niedrigen und mittleren Umsätzen folgen die Prognosen den echten Werten gut.



Extremwerte bleiben schwierig

Größere Abweichungen entstehen vor allem bei sehr hohen Umsätzen.



RMSE wird erklärbar

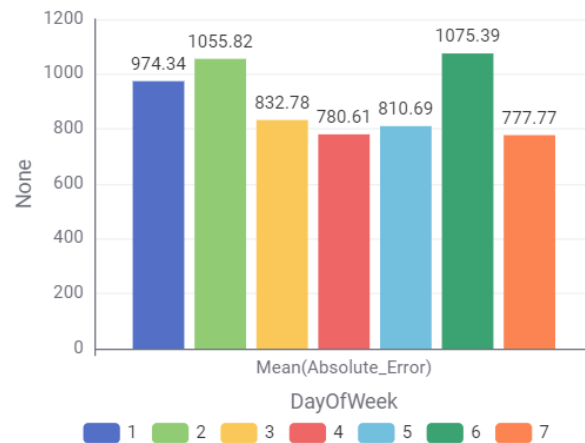
Wenige große Fehler ziehen den RMSE stärker nach oben als den MAE.

Fehler nach Gruppen

Wir prüfen, ob bestimmte Tage oder Filialtypen schwieriger vorherzusagen sind.

Nach Wochentag

Mittlerer absoluter Fehler nach Wochentag



Nach Filialtyp

Mittlerer absoluter Fehler nach Filialtyp



Zeitliche Muster

Die Fehler können je nach Wochentag leicht variieren.



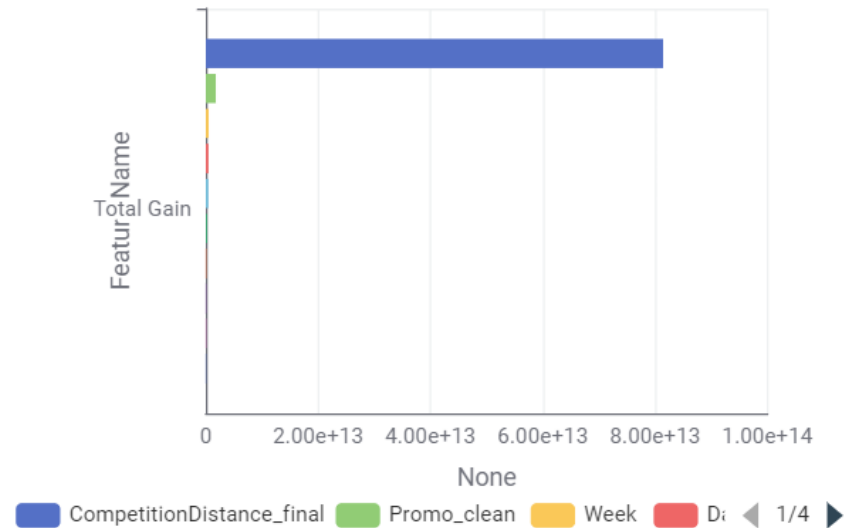
Filialtypen

Bestimmte Filialtypen sind schwieriger vorherzusagen.

Erklärung des Modells

Feature Importance zeigt, welche Faktoren XGBoost besonders stark nutzt.

Wichtigste Einflussfaktoren laut XGBoost



CompetitionDistance_final

Die Entfernung zur Konkurrenz ist der wichtigste Einflussfaktor.



Promo_clean

Promotionen verändern das normale Umsatzverhalten deutlich.



Zeitliche Variablen

Wochentag, Monat und Kalenderwoche bilden Nachfrage-Muster ab.

Modelllogik

Das Modell nutzt nicht nur Promotionen und Kalenderinformationen, sondern auch strukturelle Filialmerkmale.

07

Clustering der Filialen

Filialen mit ähnlichem Umsatzverhalten wurden gruppiert.

Warum Clustering?

Nach der Prognose wurde geprüft, ob Filialen ähnliche Gruppen bilden.



Nicht jede Filiale funktioniert gleich

Umsatzniveau, Kundenzahl, Schwankung und Promo-Effekt können sich je nach Filiale deutlich unterscheiden.



Relevanz für das Modell

Wenn klare Gruppen entstehen, kann ein gemeinsames XGBoost-Modell an Grenzen kommen.



Idee

Filialen werden nicht einzeln erklärt, sondern nach ähnlichem Verhalten zusammengefasst.

Fragestellung

Reicht ein gemeinsames Modell für alle Filialen aus, oder zeigen bestimmte Filialgruppen ein eigenes Verhalten?

Ziel: bessere fachliche Interpretation der Umsatzprognose

KNIME-Workflow: Merkmale pro Filiale

Für das Clustering wurde jede Filiale auf eine Kennzahlen-Zeile verdichtet.

Schritt	Umsetzung
Filter	nur geöffnete Tage, Store 765 entfernt
Aggregation	Kennzahlen pro Store berechnet
Merkmale	Umsatz, Kunden, Umsatz pro Kunde, Sales-CV, Promo-Lift
Clustering	Normalisierung und k-Means mit $k = 3$



Warum normalisieren?

Die Kennzahlen haben unterschiedliche Skalen. Ohne Normalisierung würde der Umsatz die anderen Merkmale zu stark dominieren.



Warum $k = 3$?

Drei Cluster waren gut lesbar und fachlich interpretierbar.

Aus Tagesdaten wurden Filialprofile

Ergebnis: drei Cluster

Die Gruppen unterscheiden sich vor allem beim durchschnittlichen Umsatz.

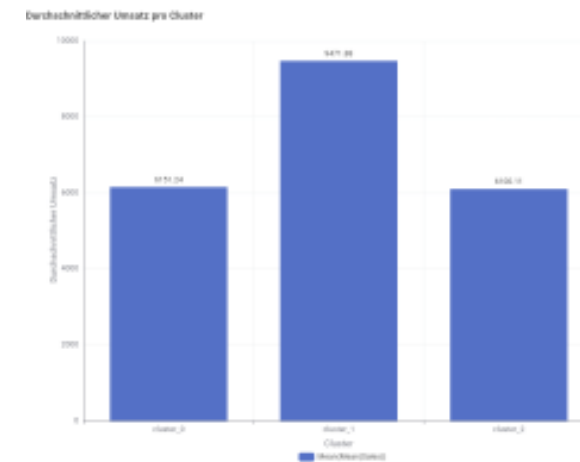
Cluster	Durchschnittlicher Umsatz	Anzahl Filialen
cluster_0	6.151	199
cluster_1	9.472	99
cluster_2	6.100	101



Kernaussage

cluster_1 hebt sich klar ab: weniger Filialen, aber deutlich höherer durchschnittlicher Umsatz.

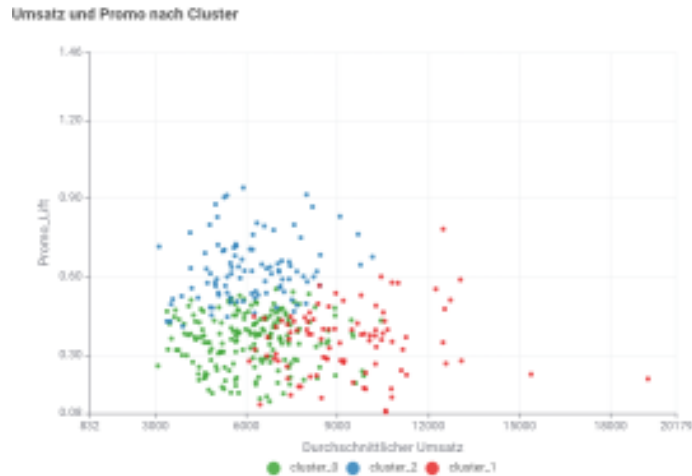
Durchschnittlicher Umsatz pro Cluster



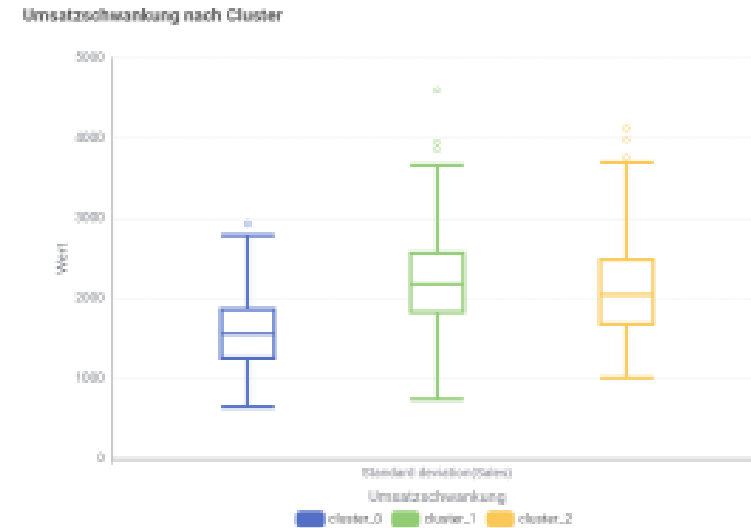
Visualisierung der Cluster

Scatter Plot und Boxplot zeigen die Trennung und die Streuung der Filialgruppen.

Umsatz und Promo-Lift nach Cluster



Umsatzschwankung nach Cluster



Interpretation

cluster_0 und cluster_2 liegen beim Umsatzniveau nah beieinander. Die Trennung entsteht stärker über Schwankung und Promo-Effekt.

Bedeutung für die Umsatzprognose

Die Cluster helfen, die Modellgrenzen fachlich besser einzuordnen.



Gemeinsames Modell

Das XGBoost-Modell erreicht insgesamt eine starke Prognosequalität und ist als Grundlage sinnvoll.



Unterschiedliche Filialgruppen

Besonders die umsatzstarken Filialen zeigen ein eigenes Verhalten.



Nächster Schritt

Man könnte zusätzlich getrennte Modelle pro Cluster testen und mit dem gemeinsamen Modell vergleichen.

Antwort

Ein gemeinsames Modell ist sinnvoll, aber die Cluster zeigen, wo differenziertere Prognosemodelle später interessant wären.

Grenzen und Fazit

Der gesamte Workflow zeigt den Weg von Rohdaten zu interpretierbaren Ergebnissen.



Grenzen

Einige Bereinigungen waren regelbasiert. Externe Faktoren wie Wetter, lokale Ereignisse oder Konkurrenzaktionen waren nicht im Datensatz enthalten.



Prognose

Das Modell liefert gute Ergebnisse, hat aber bei sehr hohen und ungewöhnlichen Umsätzen größere Schwierigkeiten.



Was erreicht wurde

Die Daten wurden geprüft, bereinigt, zusammengeführt und mit DQ-Kennzahlen bewertet.



Modellierung

Mit XGBoost wurde eine starke Umsatzprognose aufgebaut und über Fehleranalysen erklärt.



Clustering

Die Filialen wurden gruppiert, um Unterschiede zwischen Filialen sichtbar zu machen.

Fazit: Aus fehlerhaften Rohdaten wurde ein stabiler Analysedatensatz, eine starke Prognose und eine fachlich interpretierbare Sicht auf unterschiedliche Filialgruppen.